Ingeniería
en Desarrollo de Software

Actividad | 1 | Instalación Android

Studio y Diseño de aplicación

Nombre del curso

Ingeniería en Desarrollo de Software

TUTOR: Eduardo Israel Castillo García.

ALUMNO: Uziel de Jesús López Ornelas.

FECHA: 25 de marzo de 2026.

Tabla de contenido

Introducción **1**

Desarrollo **1**

Interfaz..... 1

Codificación..... 4

Prueba de la aplicación 13

Link de GitHub..... 13

Conclusión **13**

Introducción

En el primer escenario se nos pide desarrollar una aplicación de software que permita manejar una galería de sonidos, estos a su vez serán configurados en Android Studio para configurarlo y que este tenga un uso correcto. Como ya se mencionó anteriormente tenemos que realizar la instalación de Android Studio según el tutorial blindado por la sección de la materia, después de ello una vez que se ha instalado tenemos que abrir el programa, ya dentro tenemos que crear un emulador para poder visualizar el proceso de la aplicación, el documento existen diferentes archivos que nos dicen Cómo hacer la instalación del emulador o cómo crear nuestra primera aplicación sencillos pasos, después de ello tenemos que realizar la codificación la interfaz y la prueba de la aplicación, todo esto de diversas maneras, usando un emulador, o una que simple pero más rápida utilizando nuestro dispositivo Android como un puente entre los dos.

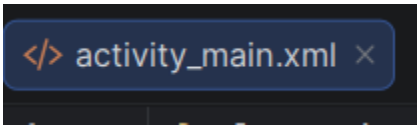
Por último, se nos pide guardar nuestro documento PDF y también el formato de la carpeta comprimida del código que creamos en la aplicación de Android Studio, este documento es el primero de tres actividades que tendremos a lo largo de la materia.

Desarrollo

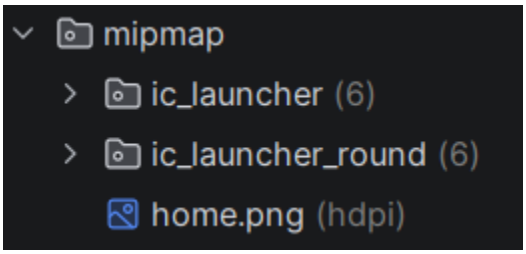
Interfaz

Abrir Android Studio:

Dentro de activity_main crear la interfaz necesaria:



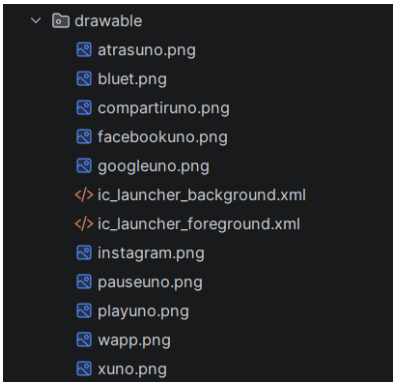
Se usarán diferentes imágenes para nuestro logo:



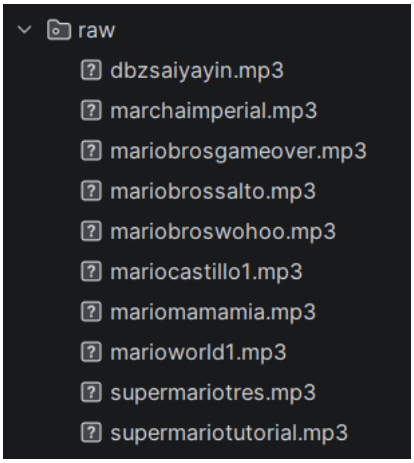
Se adaptará para que este sea consistente:

Para pasar a la siguiente pantalla se crea una nueva “Activity” y en este el nombre de “view_activity”:

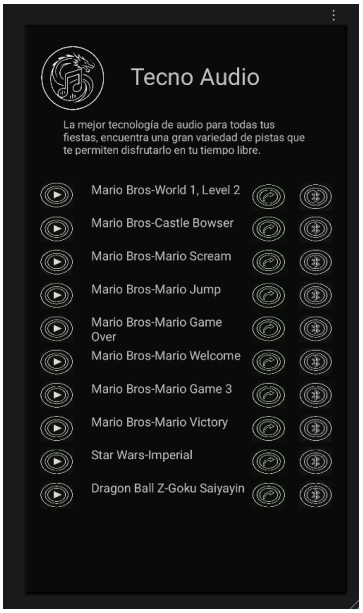
Meteremos las imágenes que necesitaremos para el lienzo actual:



Con ello también agregaremos los audios que queremos (en este caso serán diez):



Con todo hecho la interfaz se vería así:



Codificación

Para la interfaz inicial se realizó el siguiente código donde se acomoda y se organiza las capas de los diferentes elementos que conforman a pantalla de bienvenida o apertura:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android=
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#0B0A0A"
    tools:context=".MainActivity">
```

```
<ImageView
    android:id="@+id/logo_image_view"
    android:layout_width="192dp"
    android:layout_height="333dp"
    android:layout_marginStart="100dp"
    android:layout_marginTop="80dp"
    android:layout_marginEnd="100dp"
    android:layout_marginBottom="80dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:srcCompat="@mipmap/home" />
```

```
<TextView
    android:id="@+id/audio_textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="26dp"
    android:layout_marginBottom="106dp"
    android:fontFamily="sans-serif"
    android:text="Tecno Audio"
    android:textColor="#C3C3C3"
    android:textSize="48sp"
    android:textStyle="bold"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.496"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/logo_image_view
    tools:ignore="MissingConstraints" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Para darle movimiento a los iconos lo que se hace es una nueva “Android resource file”, donde tenga colocado el tipo de animación y la función que se quiere ejecutar:

desplazamiento_abajo.xml

Dentro de este se coloca la acción:

```xml
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android">

    <translate
        android:fromXDelta="0%"
        android:fromYDelta="-30%"
        android:duration="4000"/>

    <alpha
        android:fromAlpha=".01"
        android:toAlpha="1.0"
        android:duration="4000"/>

</set>
```

Igual con la animación contraria:

desplazamiento_arriba.xml

Con el código que se quiere ejecutar:

```xml
<?xml version="1.0" encoding="utf-8"?>
<set xmlns:android="http://schemas.android.com/apk/res/android">

    <translate
        android:fromXDelta="0%"
        android:fromYDelta="30%"
        android:duration="1000"/>

    <alpha
        android:fromAlpha="1.0"
        android:toAlpha="0.1"
        android:duration="1000"/>
</set>
```

Con esto hecho el “Main_activity” es nuestro puente para que este realice la acción del código:

MainActivity.kt

Importamos clases para que no tengamos errores:

```
package com.example.actividad_1

import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.graphics.Insets;
import android.content.Intent;
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import android.widget.ImageView;
import android.widget.TextView;
import android.view.animation.Animation;
import android.view.animation.AnimationUtils;
import android.os.Handler
import android.os.Looper
```

Definimos variables y las mandamos a llamar:

```
val animacion1 = AnimationUtils.loadAnimation( context = this,  id = R.anim.desplazamiento_arriba)
val animacion2 = AnimationUtils.loadAnimation( context = this,  id = R.anim.desplazamiento_arriba)
```

El nombre de nuestro icono y cuadro de texto mostrara las variables y la conexión con los mismos:

```
val logoImageView = findViewById<ImageView>(R.id.logo_image_view)
val audioTextView = findViewById<TextView>(R.id.audio_textView)
```

Los enlazaremos:

```
logoImageView.startAnimation(animacion1)
audioTextView.startAnimation(animacion2)
```

Con el siguiente comando daremos un salto o puente a la siguiente pagina que en este caso es “view_activity”, que desaparecerá en un lapso que nosotros le pongamos:

```
        Handler(Looper.getMainLooper()).postDelayed( r = {
            val intent = Intent( packageContext = this,  cls = ViewActivity::class.java)
            startActivity(intent)
            finish()
        },  delayMillis = 1500)
    }
}
```

Con esto terminado nos iremos a la siguiente pantalla donde tenemos la codificación de nuestro diseño:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#0B0A0A"
    tools:context=".ViewActivity">
```

```
<ImageView
    android:id="@+id/imageView"
    android:layout_width="85dp"
    android:layout_height="85dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.051"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.037"
    app:srcCompat="@mipmap/home" />
```

```
<TextView
    android:id="@+id/textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Tecno Audio"
    android:textColor="#C3C3C3"
    android:textSize="30sp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.549"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.063" />
```

```
<TextView
    android:id="@+id/textView2"
    android:layout_width="316dp"
    android:layout_height="50dp"
    android:text="La mejor tecnología de audio para todas tu
    android:textColor="#C3C3C3"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.493"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.171" />
```

```
<Button
    android:id="@+id/play1"
    android:layout_width="43dp"
    android:layout_height="37dp"
    android:foreground="@drawable/playuno"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.045"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.287" />
```

```
<TextView
    android:id="@+id/cancion1"
    android:layout_width="201dp"
    android:layout_height="39dp"
    android:text="Mario Bros-World 1, Level 2"
    android:textColor="#C3C3C3"
    android:textSize="16sp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.395"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.288" />
```

```
<Button
    android:id="@+id/share1"
    android:layout_width="43dp"
    android:layout_height="37dp"
    android:foreground="@drawable/compartiruno"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.79"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.29" />
```

```
<Button
    android:id="@+id/blue1"
    android:layout_width="43dp"
    android:layout_height="37dp"
    android:foreground="@drawable/bluet"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.956"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.29" />
```

```
<Button
    android:id="@+id/play2"
    android:layout_width="43dp"
    android:layout_height="37dp"
    android:foreground="@drawable/playuno"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.045"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.350" />
```

```
<TextView
    android:id="@+id/cancion2"
    android:layout_width="201dp"
    android:layout_height="39dp"
    android:text="Mario Bros-Castle Bowser"
    android:textColor="#C3C3C3"
    android:textSize="16sp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.395"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintVertical_bias="0.350" />
```

Y así sucesivamente hasta tener los diez elementos de play, texto, compartir y bluetooth.

Para finalizar le damos acción al botón de play para que este nos reproduzca los tonos que hemos colocado dentro de nuestro trabajo:

```
2 Usages
private lateinit var play1: Button
2 Usages
private lateinit var play2: Button
2 Usages
private lateinit var play3: Button
2 Usages
private lateinit var play4: Button
2 Usages
private lateinit var play5: Button
2 Usages
private lateinit var play6: Button
2 Usages
private lateinit var play7: Button
2 Usages
private lateinit var play8: Button
2 Usages
private lateinit var play9: Button
2 Usages
private lateinit var play10: Button
```

Después de esto crearemos variables con “mediaplayer”:

```
4 Usages
private lateinit var mp1: MediaPlayer
4 Usages
private lateinit var mp2: MediaPlayer
4 Usages
private lateinit var mp3: MediaPlayer
4 Usages
private lateinit var mp4: MediaPlayer
4 Usages
private lateinit var mp5: MediaPlayer
4 Usages
private lateinit var mp6: MediaPlayer
4 Usages
private lateinit var mp7: MediaPlayer
4 Usages
private lateinit var mp8: MediaPlayer
4 Usages
private lateinit var mp9: MediaPlayer
4 Usages
private lateinit var mp10: MediaPlayer
```

Las mandamos a llamar para que estos no causen errores:

```
setContentView(R.layout.activity_view)
play1 = findViewById<Button>(R.id.play1)
play2 = findViewById<Button>(R.id.play2)
play3 = findViewById<Button>(R.id.play3)
play4 = findViewById<Button>(R.id.play4)
play5 = findViewById<Button>(R.id.play5)
play6 = findViewById<Button>(R.id.play6)
play7 = findViewById<Button>(R.id.play7)
play8 = findViewById<Button>(R.id.play8)
play9 = findViewById<Button>(R.id.play9)
play10 = findViewById<Button>(R.id.play10)
```

También con las instancias de mediaplayer donde cada variable estará relacionada con el archivo de audio qe tenemos relacionado:

```
mp1 = MediaPlayer.create( context = this,  resid = R.raw.marioworld1)
mp2 = MediaPlayer.create( context = this,  resid = R.raw.mariocastillo1)
mp3 = MediaPlayer.create( context = this,  resid = R.raw.mariomamamia)
mp4 = MediaPlayer.create( context = this,  resid = R.raw.mariobrossalto)
mp5 = MediaPlayer.create( context = this,  resid = R.raw.mariobrosgameover)
mp6 = MediaPlayer.create( context = this,  resid = R.raw.supermariotutorial)
mp7 = MediaPlayer.create( context = this,  resid = R.raw.supermariotres)
mp8 = MediaPlayer.create( context = this,  resid = R.raw.mariobroswohoo)
mp9 = MediaPlayer.create( context = this,  resid = R.raw.marchaimperial)
mp10 = MediaPlayer.create( context = this,  resid = R.raw.dbzsaiyayin)
```

Por último, ejecutamos el botón de acción de play para que cuando este se presione reproduzca la canción, cuando este reproduzca la canción aparecerá un mensaje de “play” y cuando se presione para pausar aparecerá un mensaje que diga “pause”:

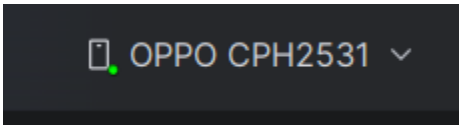
```
play1.setOnClickListener {
    if (mp1.isPlaying) {
        mp1.pause()
        Toast.makeText( context = this,  text = "Pausa",  duration = Toast.LENGTH_SHORT).show()
    } else {
        mp1.start()
        Toast.makeText( context = this,  text = "Play",  duration = Toast.LENGTH_SHORT).show()
    }
}
```

Así hasta llegar al número diez:

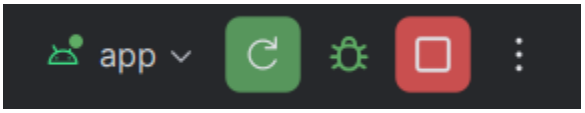
```
    play10.setOnClickListener {
        if (mp10.isPlaying) {
            mp10.pause()
            Toast.makeText( context = this,  text = "Pausa",  duration = Toast.LENGTH_SHORT).show()
        } else {
            mp10.start()
            Toast.makeText( context = this,  text = "Play",  duration = Toast.LENGTH_SHORT).show()
        }
    }
```

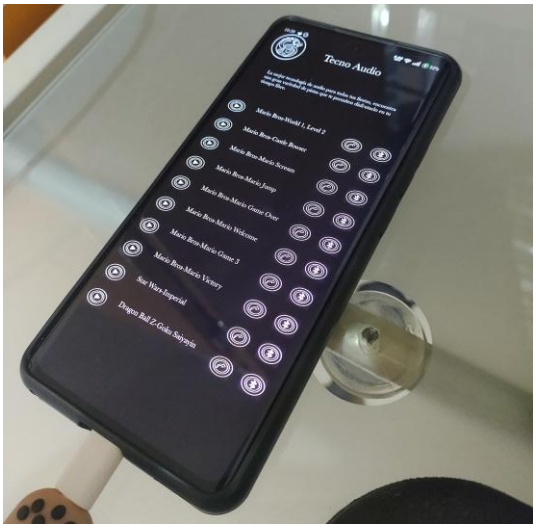
Prueba de la aplicación

Para eso conectaremos nuestro celular al emulador para ejecutar el código con lo que nos pide la actividad:



Ejecutamos la aplicación y podemos apreciar el dispositivo conectado y funcionando:





Link de GitHub

https://github.com/UZLOP984/Desarrollo_aplicaciones_moviles_II.git

Conclusión

Con esta actividad aprendí no solamente a realizar la instalación de manera adecuada del software de Android Studio, sino que también tuvimos un pequeño vistazo acerca de cómo podríamos crear una aplicación. De acuerdo al contexto o al escenario que nos pedía la actividad número uno teníamos que crear una aplicación móvil en Android que permitía manejar una galería de ringstones o tonos. Esto por supuesto creado en el lenguaje de programación Kotlin. Se nos pedía primeramente que descargáramos Android Studio lo que se hizo, fue la versión más actual para que no hubiera ningún problema con ello, se nos

entregaron manuales de investigación para crear nuestra primera aplicación o Cómo configurar dicha aplicación para tener la lista en futuros proyectos. En lo personal la primera actividad me pareció muy interesante porque manejamos la implementación de pantallas de bienvenida, puentes hacia otras pantallas, así como la integración de imágenes, iconos y sobre todo los tonos o canciones que estos iban a estar organizados dentro de la aplicación. Sinceramente esta primera actividad fue interesante ya que estamos adentrándonos cada vez más a la complejidad que maneja ese tipo de software.